Heart Disease Risk Prediction

In this machine learning project, I have collected the dataset from Kaggle (https://www.kaggle.com/ronitf/heart-disease-uci) and I will be using Machine Learning to make predictions on whether a person is suffering from Heart Disease or not.

Import libraries

Let's first import all the necessary libraries. I'll use numpy and pandas to start with. For visualization, I will use pyplot subpackage of matplotlib, use rcParams to add styling to the plots and rainbow for colors. For implementing Machine Learning models and processing of data, I will use the sklearn library.

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib import rcParams
from matplotlib.cm import rainbow
%matplotlib inline
import warnings
warnings.filterwarnings('ignore')
```

For processing the data, we'll import a few libraries. To split the available dataset for testing and training, I'll use the train_test_split method. To scale the features, I am using StandardScaler.

In [ ]:
```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

Next, we'll import all the Machine Learning algorithms I will be using.

K Neighbors Classifier

Support Vector Classifier

Decision Tree Classifier

Random Forest Classifier

In [ ]:
```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
```

Import dataset Now that we have all the libraries we will need, I can import the dataset and take a look at it. The dataset is stored in the file dataset.csv. we'll use the pandas read_csv method to read the dataset.

In [ ]:
```
dataset = pd.read_csv('heart_cleveland_upload.csv')
```

The dataset is now loaded into the variable dataset. I'll just take a glimpse of the data using the desribe() and info() methods before I actually start processing and visualizing it.

In [ ]:
```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 297 entries, 0 to 296
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   age       297 non-null    int64  
 1   sex       297 non-null    int64  
 2   cp        297 non-null    int64  
 3   trestbps  297 non-null    int64  
 4   chol      297 non-null    int64  
 5   fbs       297 non-null    int64  
 6   restecg   297 non-null    int64  
 7   thalach   297 non-null    int64  
 8   exang     297 non-null    int64  
 9   oldpeak   297 non-null    float64
 10  slope     297 non-null    int64  
 11  ca        297 non-null    int64  
 12  thal      297 non-null    int64  
 13  target    297 non-null    int64  
dtypes: float64(1), int64(13)
memory usage: 32.6 KB
```

Looks like the dataset has a total of 303 rows and there are no missing values. There are a total of 13 features along with one target value which we wish to find.

In [ ]:
```
dataset.describe()
```

Out[ ]:

	age	sex	cp	trestbps	chol	fbs	reste
count	297.000000	297.000000	297.000000	297.000000	297.000000	297.000000	297.0000
mean	54.542088	0.676768	2.158249	131.693603	247.350168	0.144781	0.9966
std	9.049736	0.468500	0.964859	17.762806	51.997583	0.352474	0.9949
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.0000
25%	48.000000	0.000000	2.000000	120.000000	211.000000	0.000000	0.0000
50%	56.000000	1.000000	2.000000	130.000000	243.000000	0.000000	1.0000
75%	61.000000	1.000000	3.000000	140.000000	276.000000	0.000000	2.0000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.0000

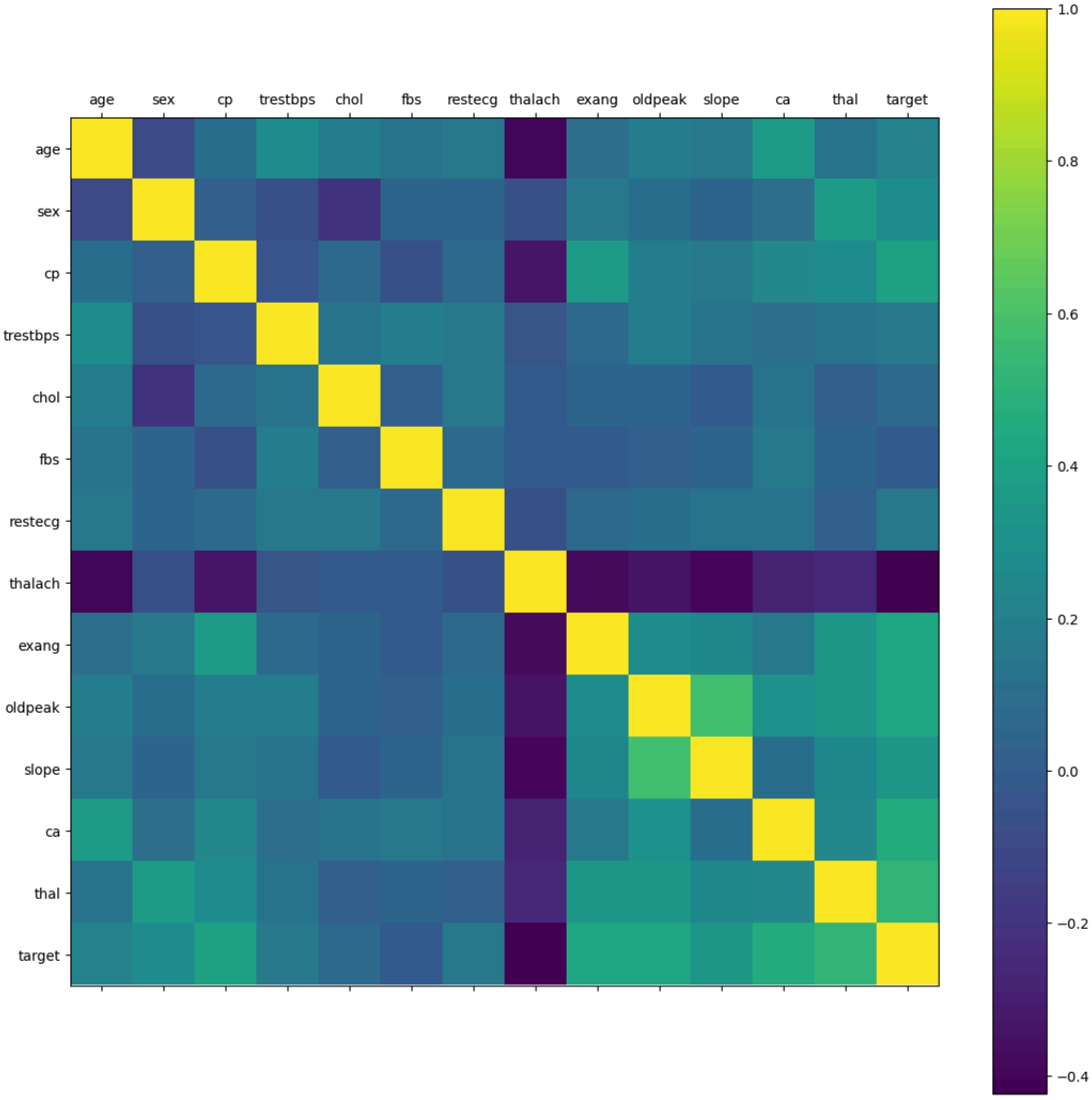
The scale of each feature column is different and quite varied as well. While the maximum for age reaches 77, the maximum of chol (serum cholestoral) is 564.

Understanding the data

Now, we can use visualizations to better understand our data and then look at any processing we might want to do.

In [ ]:
```
rcParams['figure.figsize'] = 20, 14
plt.matshow(dataset.corr())
plt.yticks(np.arange(dataset.shape[1]), dataset.columns)
plt.xticks(np.arange(dataset.shape[1]), dataset.columns)
plt.colorbar()
```

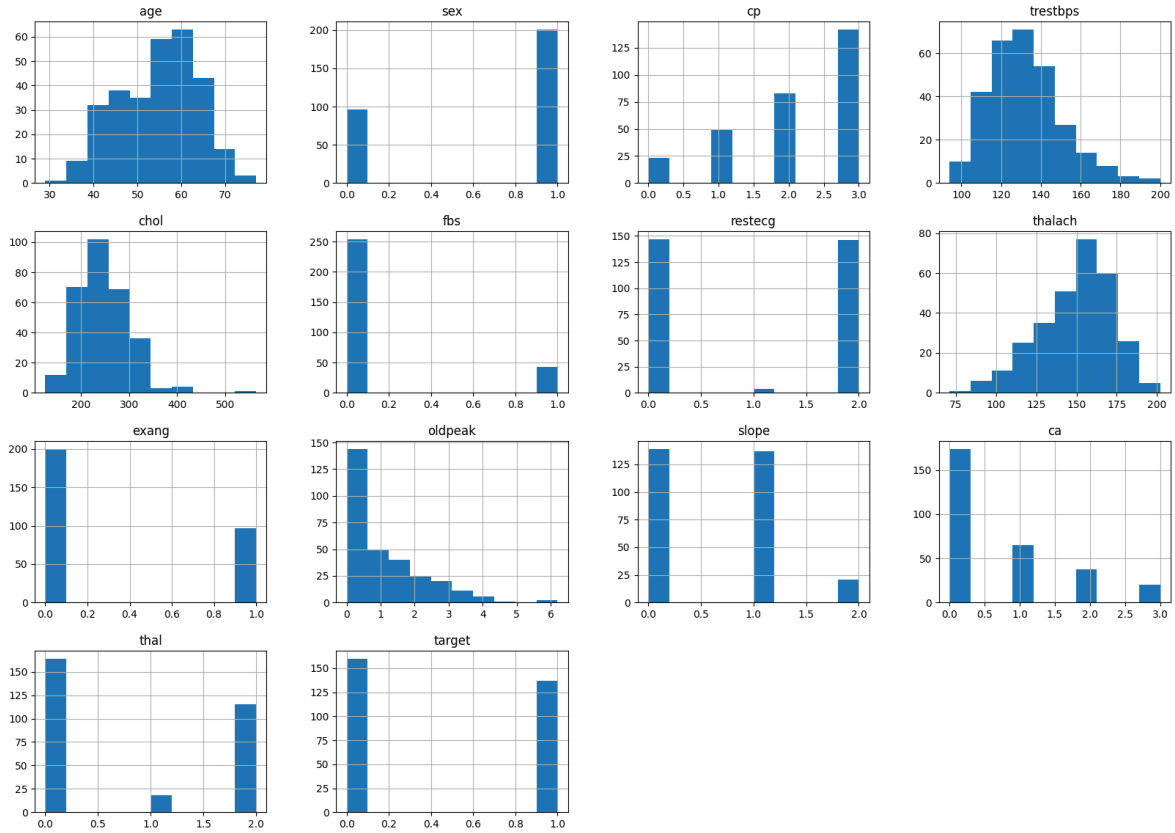
Out[ ]: <matplotlib.colorbar.Colorbar at 0x7e7ede14b9b0>



Taking a look at the correlation matrix above, it's easy to see that a few features have negative correlation with the target value while some have positive. Next, I'll take a look at the histograms for each variable.

In [ ]:
```
dataset.hist()
```

Out[ ]:
```
array([[<Axes: title={'center': 'age'}>, <Axes: title={'center': 'sex'}>,
        <Axes: title={'center': 'cp'}>,
        <Axes: title={'center': 'trestbps'}>],
       [<Axes: title={'center': 'chol'}>,
        <Axes: title={'center': 'fbs'}>,
        <Axes: title={'center': 'restecg'}>,
        <Axes: title={'center': 'thalach'}>],
       [<Axes: title={'center': 'exang'}>,
        <Axes: title={'center': 'oldpeak'}>,
        <Axes: title={'center': 'slope'}>,
        <Axes: title={'center': 'ca'}>],
       [<Axes: title={'center': 'thal'}>,
        <Axes: title={'center': 'target'}>, <Axes: >, <Axes: >]],
      dtype=object)
```



Taking a look at the histograms above, I can see that each feature has a different range of distribution. Thus, using scaling before our predictions should be of great use. Also, the categorical features do stand out.

It's always a good practice to work with a dataset where the target classes are of approximately equal size. Thus, let's check for the same.

In [ ]:
```
rcParams['figure.figsize'] = 8,6
plt.bar(dataset['target'].unique(), dataset['target'].value_counts(), color = ['
plt.xticks([0, 1])
plt.xlabel('Target Classes')
plt.ylabel('Count')
plt.title('Count of each Target Class')
```

Out[ ]:
```
Text(0.5, 1.0, 'Count of each Target Class')
```

The two classes are not exactly 50% each but the ratio is good enough to continue without dropping/increasing our data.

Data Processing After exploring the dataset, I observed that I need to convert some categorical variables into dummy variables and scale all the values before training the Machine Learning models. First, I'll use the get_dummies method to create dummy columns for categorical variables.

In [ ]:
```
dataset = pd.get_dummies(dataset, columns = ['sex', 'cp', 'fbs', 'restecg', 'exa
```

Now, I will use the StandardScaler from sklearn to scale my dataset.

In [ ]:
```
standardScaler = StandardScaler()
columns_to_scale = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']
dataset[columns_to_scale] = standardScaler.fit_transform(dataset[columns_to_scal
```

The data is not ready for our Machine Learning application.

Machine Learning

I'll now import train_test_split to split our dataset into training and testing datasets. Then, I'll import all Machine Learning models I'll be using to train and test the data.

In [ ]:
```
y = dataset['target']
X = dataset.drop(['target'], axis = 1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.33, rand
```

K Neighbors Classifier

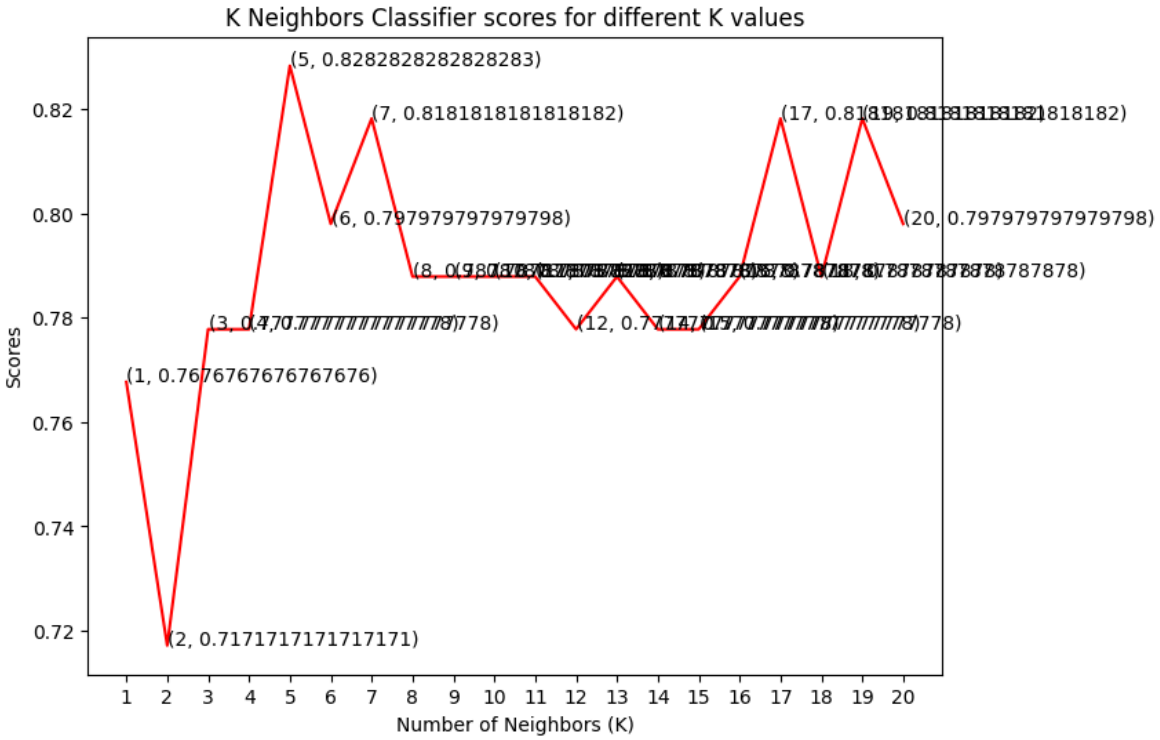
The classification score varies based on different values of neighbors that we choose. Thus, I'll plot a score graph for different values of K (neighbors) and check when do I achieve the best score.

In [ ]:
```
knn_scores = []
for k in range(1,21):
    knn_classifier = KNeighborsClassifier(n_neighbors = k)
    knn_classifier.fit(X_train, y_train)
    knn_scores.append(knn_classifier.score(X_test, y_test))
```

I have the scores for different neighbor values in the array knn_scores. I'll now plot it and see for which value of K did I get the best scores.

In [ ]:
```
plt.plot([k for k in range(1, 21)], knn_scores, color = 'red')
for i in range(1,21):
    plt.text(i, knn_scores[i-1], (i, knn_scores[i-1]))
plt.xticks([i for i in range(1, 21)])
plt.xlabel('Number of Neighbors (K)')
plt.ylabel('Scores')
plt.title('K Neighbors Classifier scores for different K values')
```

Out[ ]:
```
Text(0.5, 1.0, 'K Neighbors Classifier scores for different K values')
```



From the plot above, it is clear that the maximum score achieved was 0.87 for the 8 neighbors.

In [ ]:
```
print("The score for K Neighbors Classifier is {}% with {} nieghbors.".format(kn
```

```
The score for K Neighbors Classifier is 78.78787878787878% with 8 nieghbors.
```

Support Vector Classifier

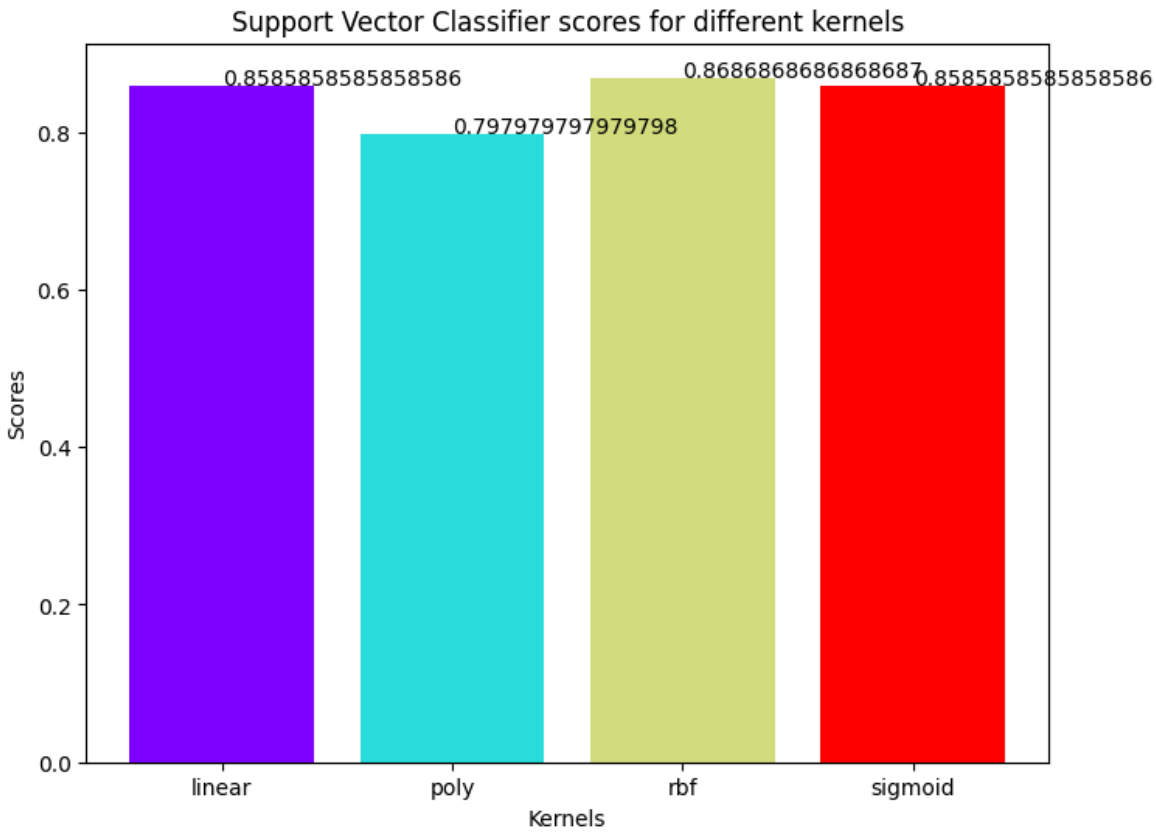
There are several kernels for Support Vector Classifier. I'll test some of them and check which has the best score.

In [ ]:
```
svc_scores = []
kernels = ['linear', 'poly', 'rbf', 'sigmoid']
for i in range(len(kernels)):
    svc_classifier = SVC(kernel = kernels[i])
    svc_classifier.fit(X_train, y_train)
    svc_scores.append(svc_classifier.score(X_test, y_test))
```

I'll now plot a bar plot of scores for each kernel and see which performed the best.

In [ ]:
```
colors = rainbow(np.linspace(0, 1, len(kernels)))
plt.bar(kernels, svc_scores, color = colors)
for i in range(len(kernels)):
    plt.text(i, svc_scores[i], svc_scores[i])
plt.xlabel('Kernels')
plt.ylabel('Scores')
plt.title('Support Vector Classifier scores for different kernels')
```

Out[ ]:
```
Text(0.5, 1.0, 'Support Vector Classifier scores for different kernels')
```



The linear kernel performed the best, being slightly better than rbf kernel.

In [ ]:
```
print("The score for Support Vector Classifier is {}% with {} kernel.".format(sv
```

```
The score for Support Vector Classifier is 85.85858585858585% with linear kernel.
```

Decision Tree Classifier Here, I'll use the Decision Tree Classifier to model the problem at hand. I'll vary between a set of max_features and see which returns the best accuracy.

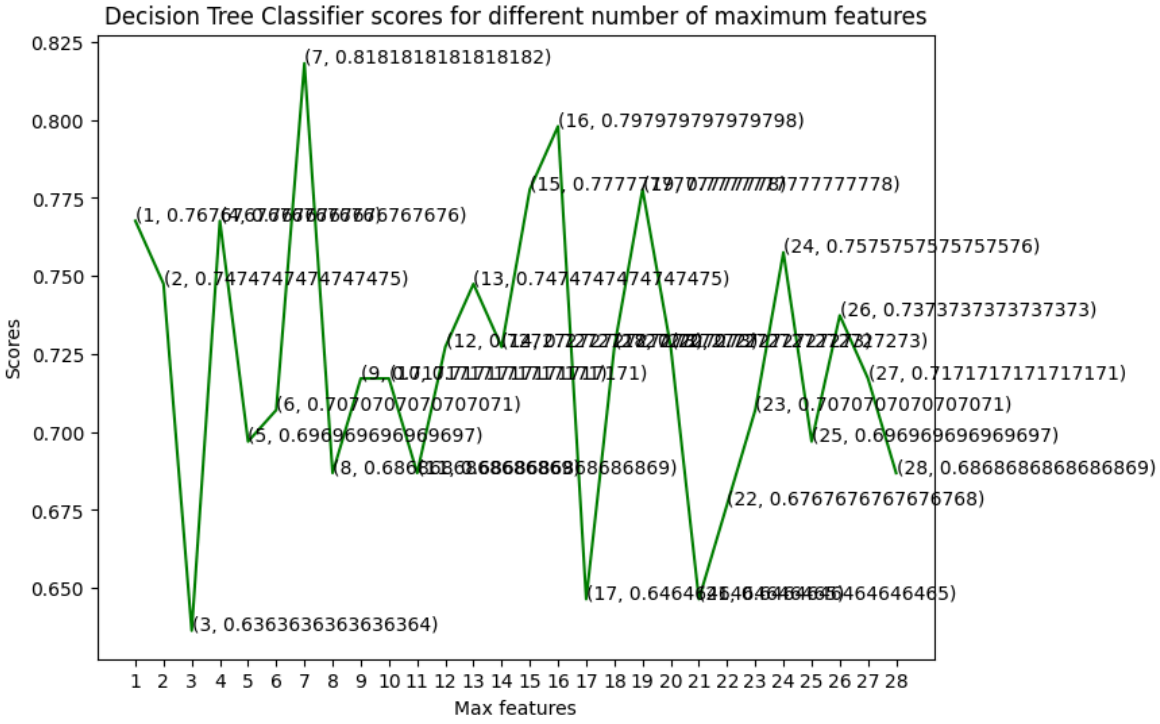
In [ ]:
```
dt_scores = []
for i in range(1, len(X.columns) + 1):
    dt_classifier = DecisionTreeClassifier(max_features = i, random_state = 0)
```

```
    dt_classifier.fit(X_train, y_train)
    dt_scores.append(dt_classifier.score(X_test, y_test))
```

I selected the maximum number of features from 1 to 30 for split. Now, let's see the scores for each of those cases.

In [ ]:
```
plt.plot([i for i in range(1, len(X.columns) + 1)], dt_scores, color = 'green')
for i in range(1, len(X.columns) + 1):
    plt.text(i, dt_scores[i-1], (i, dt_scores[i-1]))
plt.xticks([i for i in range(1, len(X.columns) + 1)])
plt.xlabel('Max features')
plt.ylabel('Scores')
plt.title('Decision Tree Classifier scores for different number of maximum featu
```

Out[ ]:
```
Text(0.5, 1.0, 'Decision Tree Classifier scores for different number of maximum
features')
```



The model achieved the best accuracy at three values of maximum features, 2, 4 and 18.

In [ ]:
```
print("The score for Decision Tree Classifier is {}% with {} maximum features.".
```

```
The score for Decision Tree Classifier is 72.72727272727273% with [2, 4, 18] maxi
mum features.
```

Random Forest Classifier

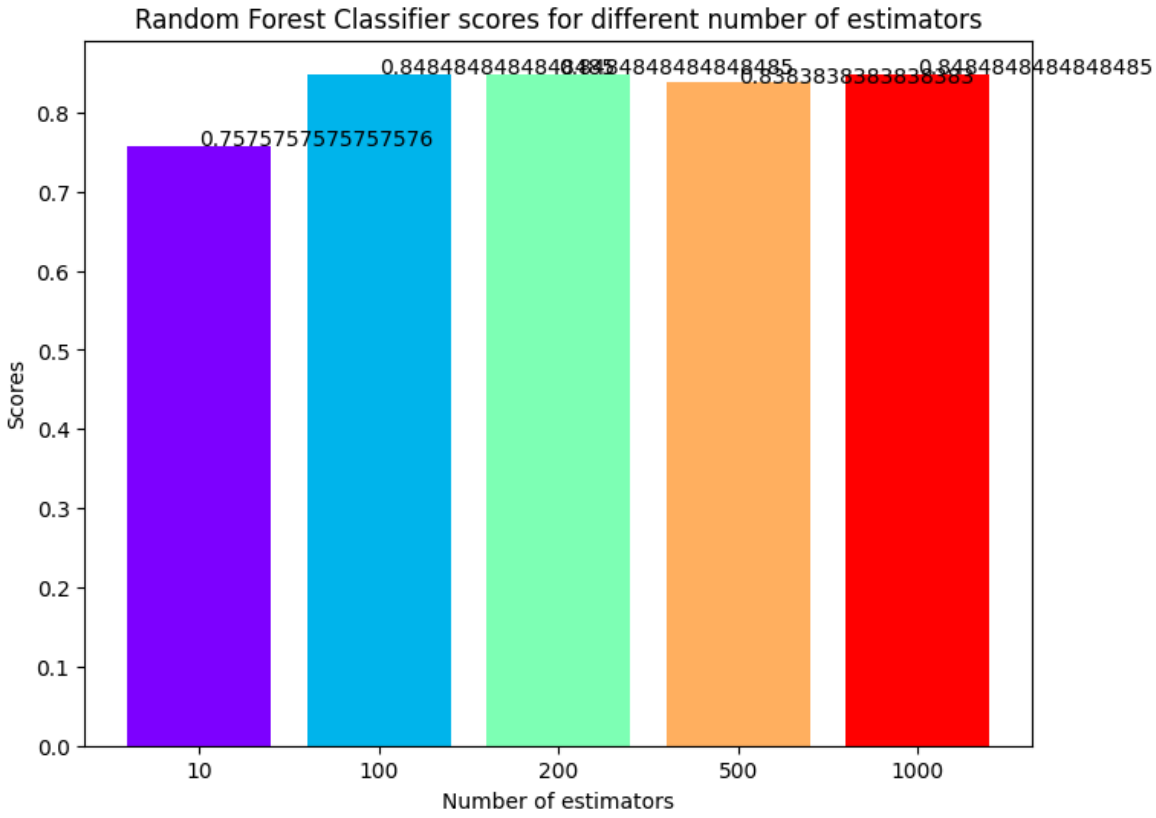
Now, I'll use the ensemble method, Random Forest Classifier, to create the model and vary the number of estimators to see their effect.

In [ ]:
```
rf_scores = []
estimators = [10, 100, 200, 500, 1000]
for i in estimators:
    rf_classifier = RandomForestClassifier(n_estimators = i, random_state = 0)
    rf_classifier.fit(X_train, y_train)
    rf_scores.append(rf_classifier.score(X_test, y_test))
```

The model is trained and the scores are recorded. Let's plot a bar plot to compare the scores.

In [ ]:
```
colors = rainbow(np.linspace(0, 1, len(estimators)))
plt.bar([i for i in range(len(estimators))], rf_scores, color = colors, width =
for i in range(len(estimators)):
    plt.text(i, rf_scores[i], rf_scores[i])
plt.xticks(ticks = [i for i in range(len(estimators))], labels = [str(estimator)
plt.xlabel('Number of estimators')
plt.ylabel('Scores')
plt.title('Random Forest Classifier scores for different number of estimators')
```

Out[ ]:
```
Text(0.5, 1.0, 'Random Forest Classifier scores for different number of estimat
ors')
```



The maximum score is achieved when the total estimators are 100 or 500.

In [ ]:
```
print("The score for Random Forest Classifier is {}% with {} estimators.".format
```

```
The score for Random Forest Classifier is 84.84848484848484% with [100, 500] esti
mators.
```

The score for Random Forest Classifier is 84.0% with [100, 500] estimators.

Conclusion

In this project, I used Machine Learning to predict risk of a person suffering from a heart disease. After importing the data, I analysed it using plots. Then, I did generated dummy variables for categorical features and scaled other features. I then applied four Machine Learning algorithms, K Neighbors Classifier, Support Vector Classifier, Decision Tree Classifier and Random Forest Classifier. I varied parameters across each model to

improve their scores. In the end, K Neighbors Classifier achieved the highest score of 84% with 8 nearest neighbors.

In [ ]: